9. Lower Bounds via Kolmogorov Complexity

The concept of Kolmogorov complexity can be used to prove complexity lower bounds. In many cases the proofs obtained in this way are much more "elegant," or at least shorter, than the original proofs. In a few cases, the lower bounds were first achieved by means of Kolmogorov complexity.

The method of using Kolmogorov complexity to establish lower bounds works as follows: Suppose we want to prove a lower bound on the running time of a Turing machine to perform a certain task – or a lower bound on the size of some other mathematical or computational object. Let x be a sufficiently long Kolmogorov random string, i.e., $K(x) \geq |x|$. Now we assume that the lower bound we are seeking is violated, e.g., there is a Turing machine that performs the given task more quickly than the stated bound. Then perhaps there is a way to use this Turing machine – and possibly some additional information – to describe the Kolmogorov random string x with fewer than $n = |x|$ bits. This would, of course, be a contradiction and establish the lower bound.

Our first example of this approach comes from number theory rather than computer science, but the result has implications for computer science as well. We will use Kolmogorov complexity to prove that there are infinitely many prime numbers. In fact, the argument we will give can even be extended to prove a weak form of the Prime Number Theorem.

So suppose that there are only finitely many prime numbers, say $p_1, p_2, \ldots, p_k$. Let n be a sufficiently large number so that the Kolmogorov complexity of the binary representation of n is not compressible, i.e., $K(bin(n)) \geq |bin(n)| = \log n$.[1] Every natural number has a unique prime factorization, i.e., $n = p_1^{n_1} p_2^{n_2} \ldots p_k^{n_k}$. So the numbers $n_1, n_2, \ldots, n_k$ (coded as bit strings) constitute a unique description of n, and thus n – or rather $bin(n)$ – can be reconstructed from the sequence $n_1, n_2, \ldots, n_k$.

[1] For simplicity in this example we will do computations assuming that $|bin(n)| = \log n$. The exact value is actually $|bin(n)| = \begin{cases} 1, & n = 0 \\ \lfloor \log_2 n \rfloor + 1, & n > 0 \end{cases}$. But since additive constants do not play a role in this context, this simplification does not change the validity of the argument.

U. Schöning et al., *Gems of Theoretical Computer Science*
© Springer-Verlag Berlin Heidelberg 1998

Exercise 9.1. Use the observations just made to determine an upper bound on $K(bin(n))$ that contradicts the choice of n made above and thus completes the proof that there are infinitely many primes. ◁

This argument can be pushed farther. The *Prime Number Theorem* is a famous and hard theorem in number theory which says that $\pi(n)$, the number of prime numbers $\leq n$, is asymptotic to $n/\ln n$, that is

$$\pi(n) \sim \frac{n}{\ln n}.$$

With significantly simpler arguments – using Kolmogorov complexity – we will show that for infinitely many n a lower bound of

$$\pi(n) \geq \frac{n}{\log^2 n}$$

holds. This lower bound is sufficient for many applications.

Let p_m be the mth prime number. It is sufficient to show that for infinitely many m, $p_m \leq m \log^2 m$.

Exercise 9.2. Show that this is indeed sufficient. ◁

Now let n be a sufficiently large natural number such that $K(bin(n)) \geq \log n$. Let p_m be the largest prime number that divides n. The number n can be (algorithmically) reconstructed from m and n/p_m. So a bit string that codes these two numbers is a sufficient description of n.

Exercise 9.3. Give an algorithm that reconstructs n from the numbers m and $k = n/p_m$. ◁

Thus

$$\log(n) \leq K(bin(n)) \leq |\text{encoding of } m \text{ and } n/p_m| .$$

The problem is that we cannot simply write down the binary representations of m and n/p_m one after the other, since then we would not know where one number ended and the other began. We must sacrifice some additional (and in this case very costly) bits to make our encoding such that it can be correctly decoded.

For a string $w = a_1 a_2 \ldots a_{n-1} a_n \in \{0,1\}^*$ let $\overline{w} = a_1 0 a_2 0 \ldots a_{n-1} 0 a_n 1$. By means of this encoding of w, the end of the code for w can be recognized by the final '1'. Such a code is called *self-terminating*. More formally, an encoding scheme is self-terminating if it can "recognize its own end" in the following sense: From any string of the form code$(w)v$, where v is arbitrary, it is possible to recover code(w) algorithmically.

We could code the pair of numbers as $\overline{bin(m)} bin(n/p_m)$. But since $|\overline{w}| = 2|w|$, this would cost $2\log m + \log(n/p_m)$ bits. Plugging this into the approximation above would yield: $\log n \leq K(bin(n)) \leq 2\log m + \log n - \log p_m$, so $p_m \leq m^2$, i.e., $\pi(n) \geq \sqrt{n}$. So this method of coding wastes too many bits.

If, however, we let

$$\text{code}(w) = \overline{bin(|w|)}w \ ,$$

then we get another coding of w that is also self-terminating.

Exercise 9.4. Justify the claim just made. Furthermore, compute the length of code(w) as a function of w. Use this to show that $p_m \leq m \log^2 m$. ◁

Exercise 9.5. The improvement from $\overline{w}$ to code(w) can be iterated. What self-terminating code does one get in this way? What improved lower bound for $\pi(n)$ does this yield? ◁

It is interesting to note that any improvement in the length (or, rather, shortness) of self-terminating codes can be translated directly into a sharper bound in our 'Weak Prime Number Theorem.'

Exercise 9.6. Show that if n is a sufficiently large number and its binary representation has high Kolmogorov complexity, (i.e., $K(bin(n)) \geq \log n$), then n cannot be a prime number.

Hint: Use $\pi(n) \sim n/\ln n$. ◁

* * * * *

This Kolmogorov method can also be used in the context of circuit complexity. Consider a boolean function f from $\{0,1\}^n$ to $\{0,1\}$. The circuit complexity of such a function is the smallest number of boolean gates that suffice to build a circuit that has n (binary) input gates and computes the function f. Since a truth table for such a function has 2^n rows, by giving a binary string of length 2^n (the evaluation vector) such a function can be uniquely characterized.

Now we select an especially "difficult" boolean function, f, for which the Kolmogorov complexity of this string is high, i.e., $K \geq 2^n$. What is the circuit complexity of the associated boolean function f? Since a boolean circuit completely characterizes a function (and, in turn, its encoding as a binary string), we have the following inequality for the Kolmogorov complexity of the binary string for f:

$$K \leq |\text{the shortest description of the circuit}| + O(1) \ .$$

How many bits do we need to represent a circuit consisting of g gates? The circuit can be described by listing all its gates. Each gate is described by giving its type (i.e., which boolean operation it computes) and the numbers of the gates (or inputs) that feed into the gate. This requires $c + 2\log(g+n)$ bits for each gate.

Exercise 9.7. Let size(f) be the circuit complexity of f, that is, the smallest number of gates that can be used to compute f. Show that size(f) $= \Omega(2^n/n)$. ◁

Exercise 9.8.$^\diamond$ A formula is a circuit in which every gate has fan-out 1. Show that the smallest formula for f has size $\Omega(2^n/\log n)$.

Hint: A formula can be encoded especially compactly, without the use of any parentheses, in "reverse Polish notation." ◁

* * * * *

Finally, we want to use the Kolmogorov method to prove that any one-tape Turing machine that accepts the language

$$L = \{w0^{|w|}w \mid w \in \{0,1\}^*\}$$

requires at least quadratic ($\Omega(n^2)$) time. From this it follows (or at least is provable with only slight modifications) that the languages

$$\{ww \mid w \in \{0,1\}^*\}$$

and

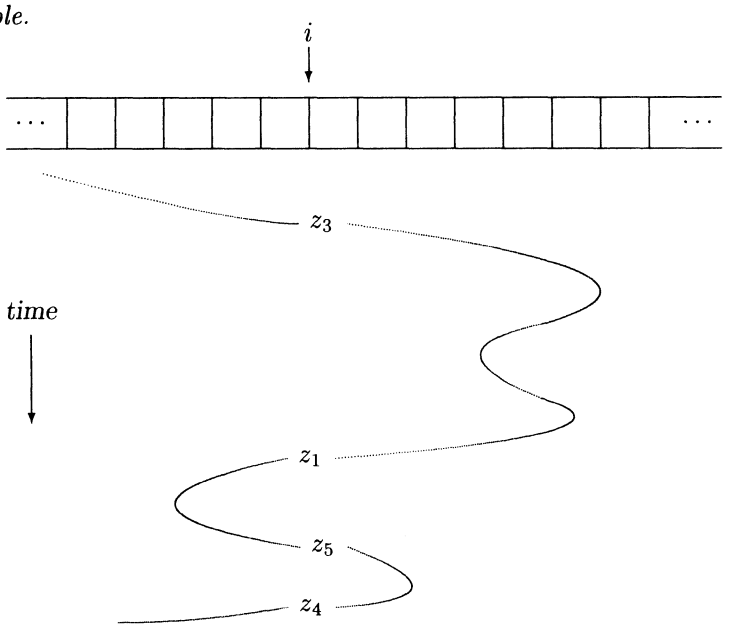
$$\{ww^R \mid w \in \{0,1\}^*\}$$

also require quadratic time. In the latter, a^R is the reversal of a:

$$(a_1 \ldots a_n)^R = a_n \ldots a_1 \ .$$

We use the concept of a crossing sequence. Let M be a Turing machine and let i be some fixed boundary between two adjacent cells on the tape. On a given input x, the crossing sequence for x and i is the sequence of states in which the Turing machine finds itself as its read-write head crosses point i on the tape. We denote this crossing sequence by $CS_M(x,i)$. $CS_M(x,i)$ is an element of Z^*, where Z is the set of states for the Turing machine.

Example.



$$CS = z_3 z_1 z_5 z_4$$

Exercise 9.9. Why is

$$\sum_{i=-\infty}^{\infty} |CS_M(x,i)| = time_M(x) \ ?$$

($time_M(x)$ is the running time of M on input x.) ◁

By Exercise 9.9, in order to show that $time_M(x) = \Omega(n^2)$, it is sufficient to show that a number of crossing sequences $CS_M(x,i)$ for x (specifically $\Omega(n)$ of them) each have length at least $\Omega(n)$. We will restrict our attention to the crossing sequences that occur in the middle $n/3$ positions of the input, where n is the total length of the input. This is the portion that on input $w0^{|w|}w$ consists entirely of 0's.

Without loss of generality, we can assume that all of our Turing machines only enter the halting state when their read-write head is on cell 0 of the tape.

Exercise 9.10.° Prove the following lemma:

Let $|x| = i$. If $CS_M(xy,i) = CS_M(xz,i)$, then $xy \in L(M)$ if and only if $xz \in L(M)$. ◁

Exercise 9.11. At what point in the proof of the previous exercise do we make use of the "WLOG" assumption above? ◁

Exercise 9.12.$^{\diamond}$ Prove the following lemma:

If $i = |x| = |x'|$ and $c = CS_M(xy, i) = CS_M(x'y', i)$, then $c = CS_M(xy', i) = CS_M(x'y, i)$. ◁

Now let M be a Turing machine that accepts L. Let $w0^{|w|}w$ be an input for M and let $|w| \leq i \leq 2|w|$. By the previous lemma we know that for distinct w and w' with $|w| = |w'|$, the crossing sequences $CS_M(w0^{|w|}w\,,\, i)$ and $CS_M(w'0^{|w'|}w'\,,\, i)$ must also be distinct.

Exercise 9.13. Why? ◁

Exercise 9.14. Describe an algorithm that on inputs M, $m \in \mathbb{N}$, $i \in \mathbb{N}$, $m \leq i \leq 2m$ and crossing sequence c (all coded in binary), outputs the string w of length m for which $CS_M(w0^{|w|}w\,,\, i) = c$. (By the previous observation, w, if it exists, must be unique.) ◁

We conclude, therefore, that the Kolmogorov complexity of w must satisfy $K(w) \leq O(\log |w|) + |c|$. If w is chosen to be Kolmogorov random – here's where the Kolmogorov argument comes in – then $K(w) \geq |w|$, so $|c| \geq |w| - O(\log n)$.

Exercise 9.15. Complete the proof that for every one-tape Turing machine M that accepts L, $time_M(x) = \Omega(|x|^2)$. ◁

As an aside, we mention that the argument above actually works not only for a string w that is Kolmogorov random but also for any "typical" string w. The expected value (under the usual uniform distribution) for the Kolmogorov complexity is $E(K(w)) \geq |w| - 2$ (cf. Topic 8). So even in the "average" case, $time_M(x) = \Omega(|x|^2)$.

References

For more on the Prime Number Theorem see a book on number theory, for example,

- I. Niven, H.S. Zuckerman: *An Introduction to the Theory of Numbers*, Wiley, 1960.

The following references contain examples of using Kolmogorov complexity to obtain lower bounds:

- W.J. Paul: Kolmogorov complexity and lower bounds, *Foundations of Computation Theory*, Akademie-Verlag, Berlin, 1979.
- W.J. Paul, J.I. Seiferas, J. Simon: An information-theoretic approach to time bounds for on-line computation, *Journal of Computer and System Sciences* 23 (1981), 108–126.

- M. Li, P. Vitányi: Applications of Kolmogorov complexity in the theory of computation, in A.L. Selman, ed., *Complexity Theory Retrospective*, Springer, 1990, 147–203.
- M. Li, P. Vitányi: *An Introduction to Kolmogorov Complexity and its Applications, 2nd edition*, Springer, 1997.